trainitek

Code Once, Use Everywhere: Building Shared Libraries for Multiple Projects

Vadzim Prudnikau



APOTEK 1

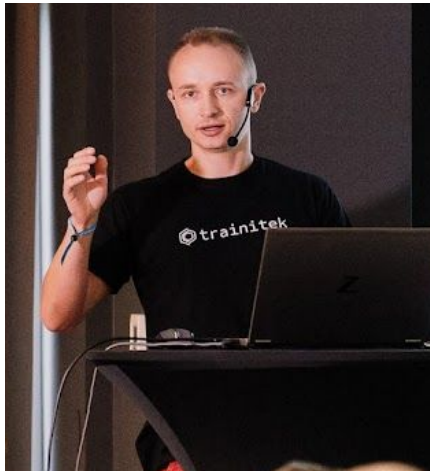
My motivation

- I developed a common library that several projects (5) have used for 4 years 💪
- I saw different approaches at work and conferences, and I felt I could do it a bit better 💡
- I would like to share my own experience 💖

trainitek

Shortly about me

I run training in Spring Boot, Architecture, DDD, TDD, etc.



Co-owner and instructor at trainitek

Hands-on architect at APOTEK 1

Vadzim Prudnikau

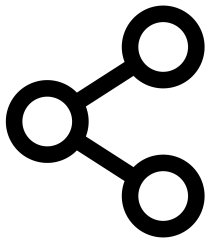
vadim-prudnikov



What will we address today?

- **When** we need **to share** our code
- **What not** to **put** into the shared code
- **Structure** of the solution **and** its **integration** with **Spring Boot** ⭐
- Documentation
- Making changes
- Forcing vs advertising

When we need to share our code



When we need to share our code

Multiple projects solve the same technical problems

&&

You are tired of copy-pasting the same solutions across projects

&&

&&

People agree to use standardised approaches

&&

The shared code costs less than repeating it

It’s not that easy

- I made mistakes myself regarding these points. It’s quite hard to follow it.

Multiple projects solve the same technical problems **&&** **You are tired of copy-pasting the same solutions across projects**

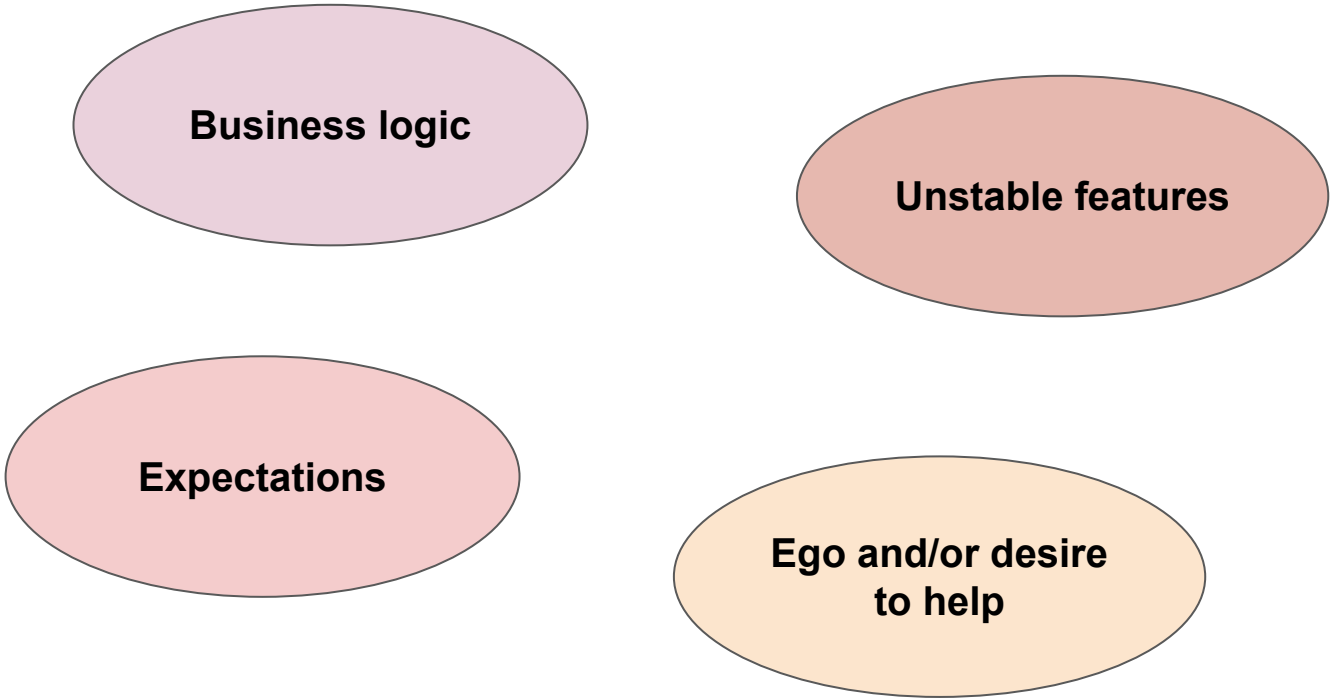
&& **&&**

People agree to use standardised approaches **&&** **The shared code costs less than repeating it**

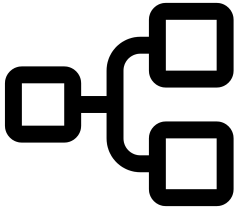
What **not** to put into the shared code



What **not** to put into the shared code



Structure of the library and its integration with Spring Boot





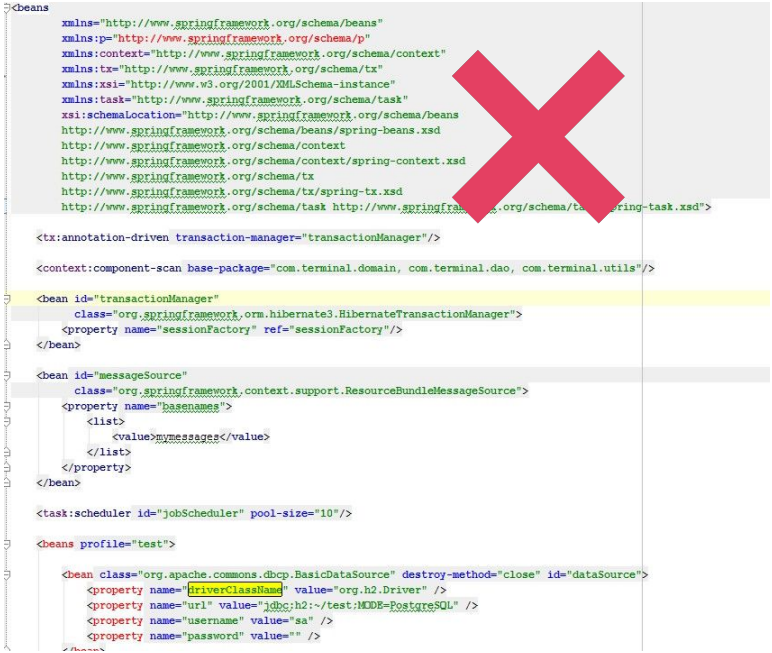
Rationale 1/5

- The library should be easy to include, ideally by adding a Maven dependency

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Rationale 2/5

- Developers should **not** be bothered by the configuration of the library



https://stackoverflow.com/questions/30578950/configure-spring-xml-configuration-namespaces-to-avoid-idea-warnings

Rationale 3/5

- It should be possible to use only specific modules

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Rationale 4/5

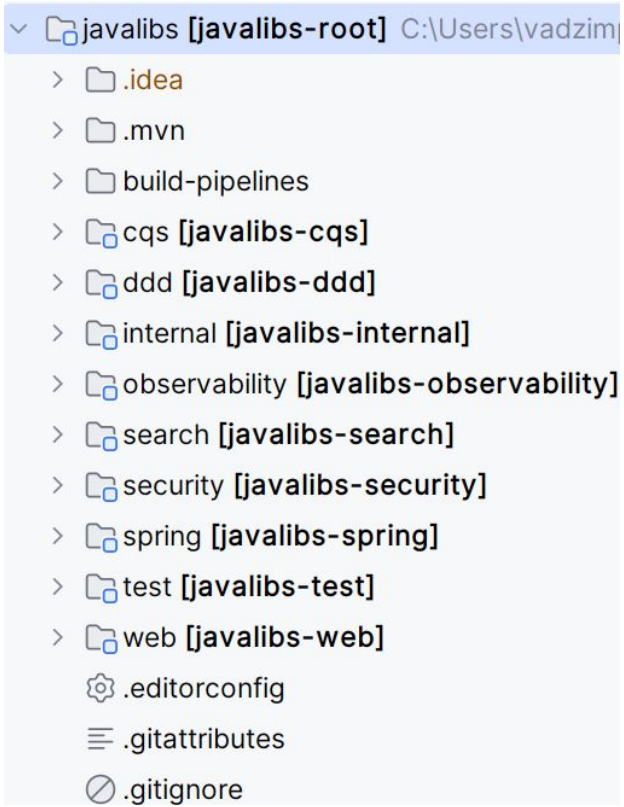
- The library structure should follow and provide some guidelines

Rationale 5/5

- Our code doesn’t always depend on Spring Framework

trainitek

Root level



APOTEK 1

trainitek

Root pom.xml

```
<groupId>no.apotek1.javalibs</groupId>
<artifactId>javalibs-root</artifactId>
<version>3.7-SNAPSHOT</version>
<packaging>pom</packaging>

<properties...>

<modules...>

<build>
    <pluginManagement...>

    <plugins...>
</build>

<dependencyManagement...>

<dependencies...>
```

Shared properties: JDK version, dependency versions, etc.

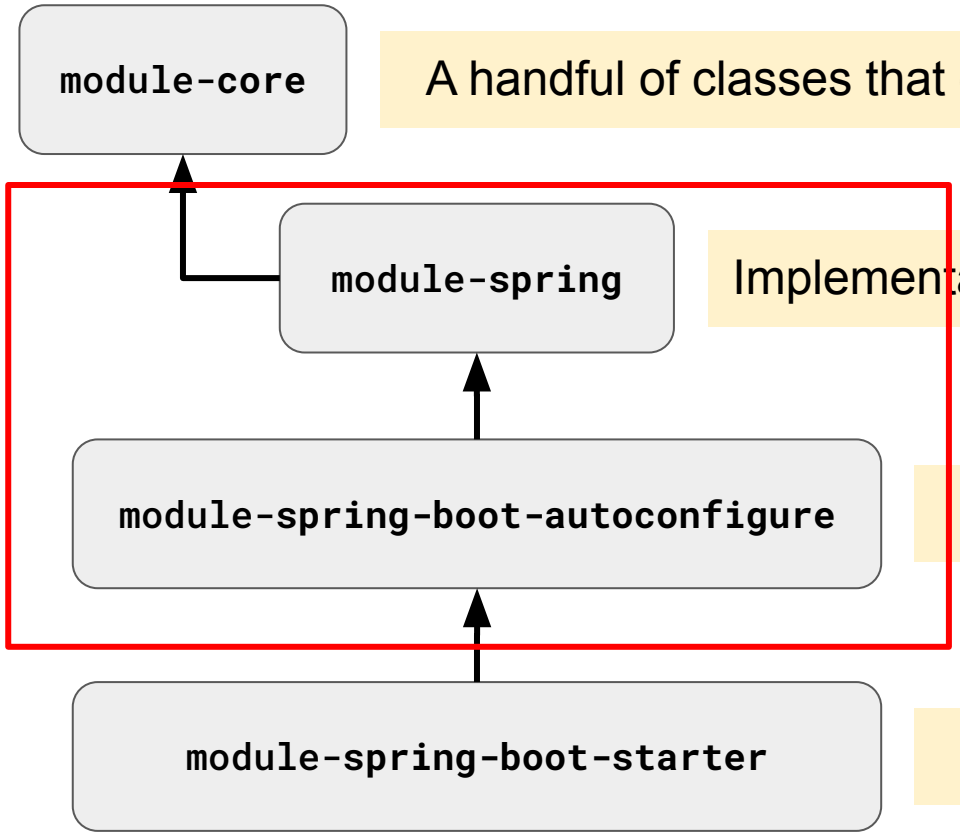
Default plugins settings: checkstyle, surefire, etc.

Default plugins to run: gmavenplus and checkstyle

Dependencies with their versions

Default dependencies like logging

Module level: concept



A handful of classes that don't depend on any specific framework

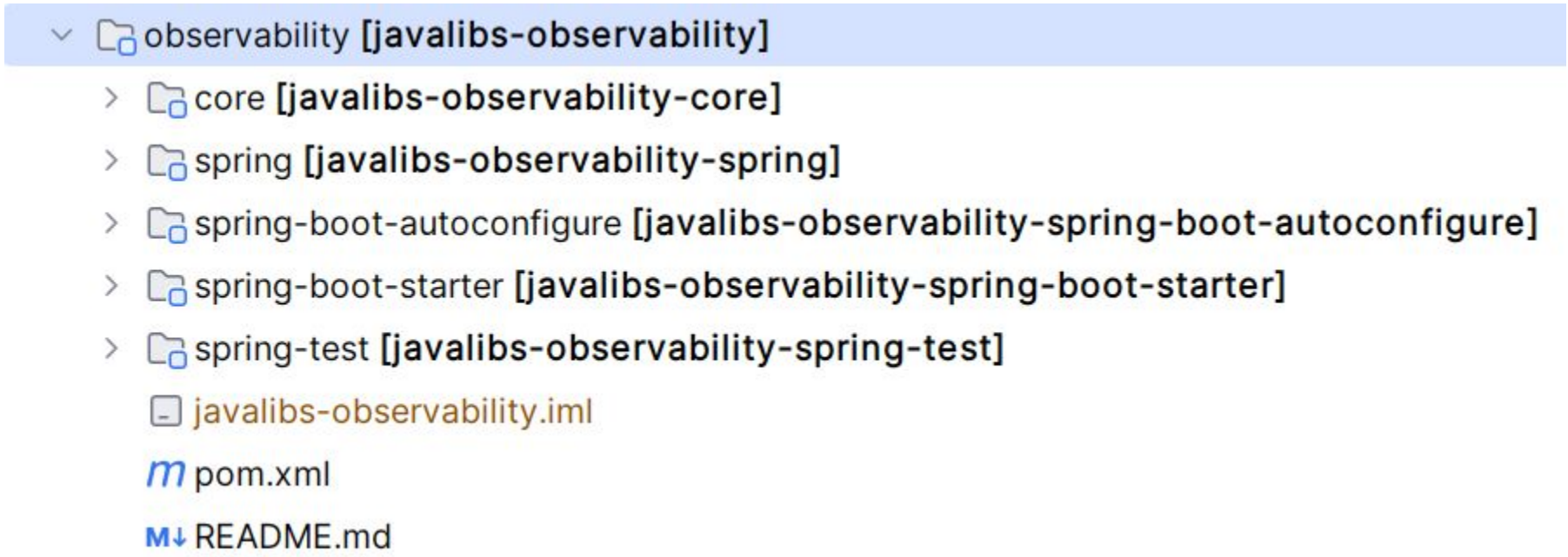
Implementations with the help of Spring Framework

This can be one module

Autoconfigurations and properties

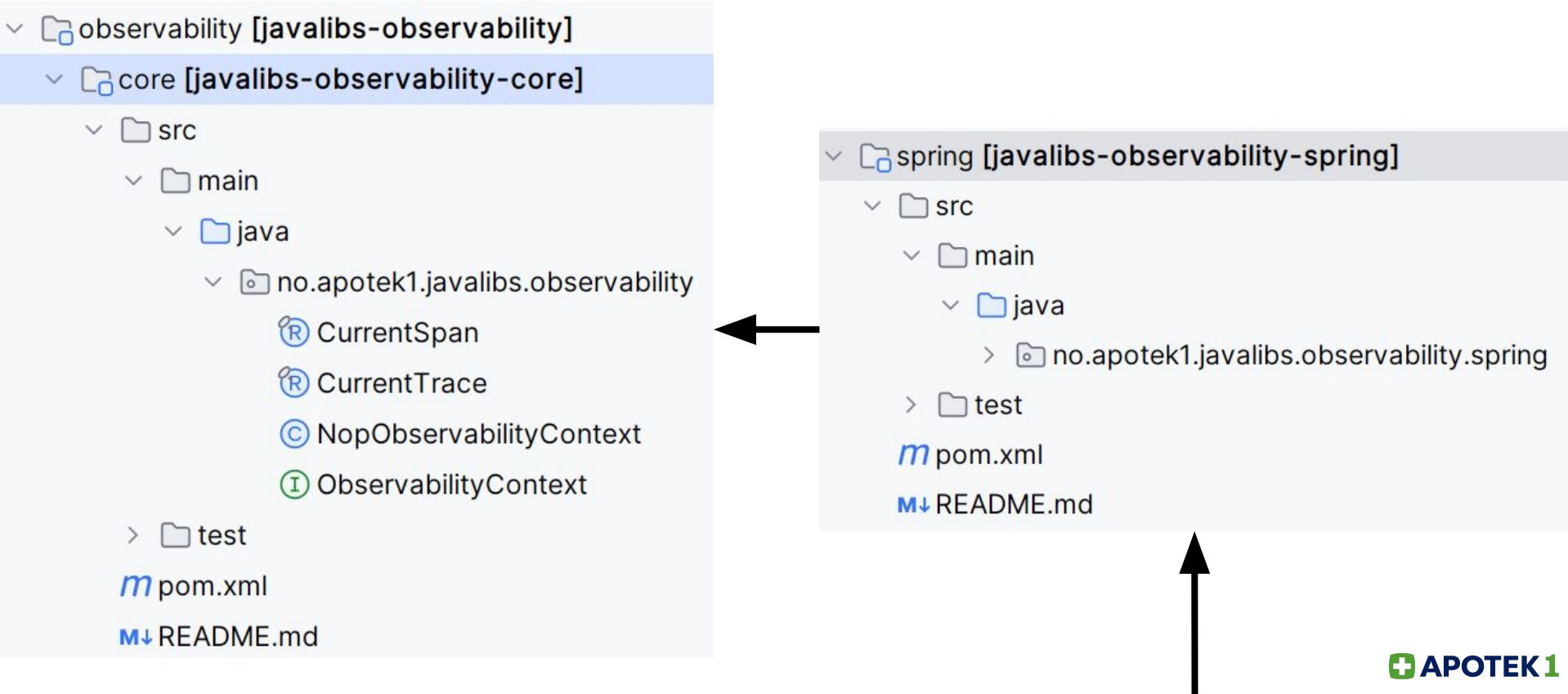
Default set of dependencies

Module level: example



trainitek

-core and -spring



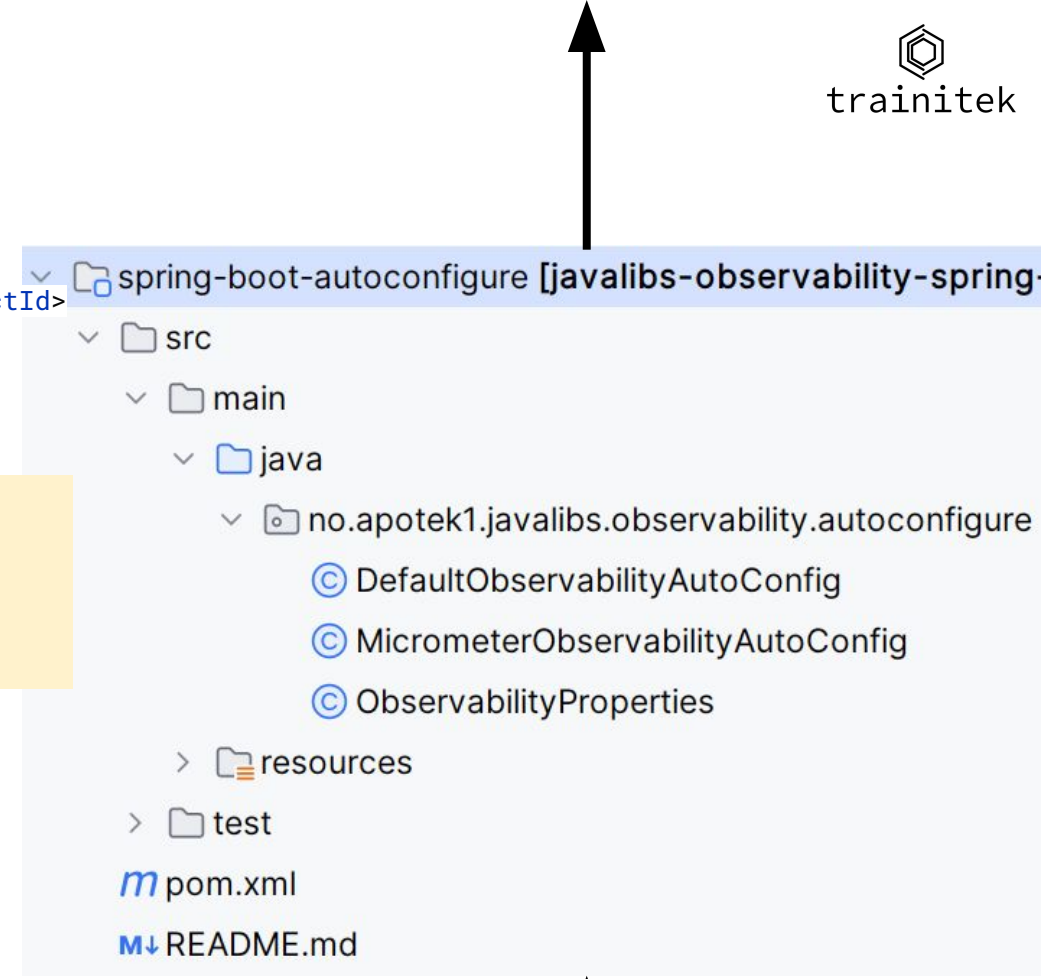
APOTEK 1

trainitek

-spring-boot-autoconfigure

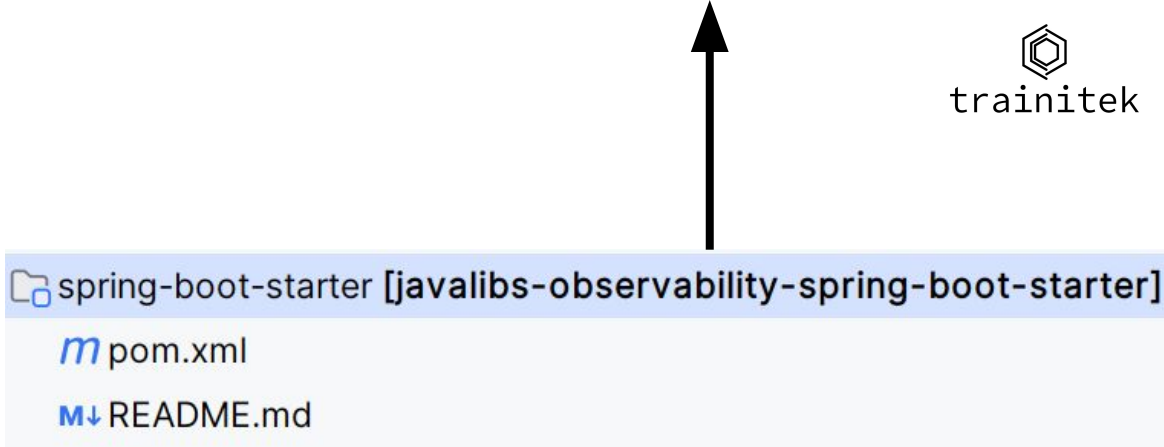
```
<dependency>
    <groupId>no.apotek1.javalibs</groupId>
    <artifactId>javalibs-observability-spring</artifactId>
    <version>${project.version}</version>
</dependency>
<dependency>
    <groupId>no.apotek1.javalibs</groupId>
    <artifactId>javalibs-test-core</artifactId>
    <v
    <s
</depe
<depen
    <g
    <a
    <v
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing</artifactId>
    <optional>true</optional>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-autoconfigure</artifactId>
</dependency>
```

Optional dependencies prevent clients from depending on unnecessary implementations!



APOTEK 1

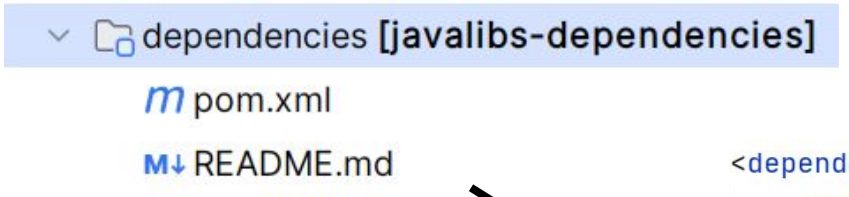
-spring-boot-starter




```xml
<dependency>
    <groupId>no.apotek1.javalibs</groupId>
    <artifactId>javalibs-observability-spring-boot-autoconfigure</artifactId>
    <version>${project.version}</version>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing</artifactId>
</dependency>
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>
```

trainitek

Special module: dependencies (BOM)



javalibs

Client project

```
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>no.apotek1.javalibs</groupId>
            <artifactId>javalibs-dependencies</artifactId>
            <version>${javalibs.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
```

Caveat: version ranges aren't supported for BOMs

```
<javalibs.version>[4.4,4.5)</javalibs.version>
```

```
<dependency>
    <groupId>no.apotek1.javalibs</groupId>
    <artifactId>javalibs-cqs-spring-boot-starter</artifactId>
</dependency>
```

APOTEK 1

Auto Configurations

```
@AutoConfiguration
@ConditionalOnClass(Tracer.class)
@EnableConfigurationProperties(ObservabilityProperties.class)
public class MicrometerObservabilityAutoConfig {

    @Bean
    @ConditionalOnMissingBean
    ObservabilityContext micrometerObservabilityContext(Tracer tracer) {
        return new MicrometerObservabilityContext(tracer);
    }
}
```

@Configuration and @AutoConfiguration

```java
@Configuration
public class MyConfig {

    @Bean
    ServiceA serviceA() {
        return new ServiceA(serviceB());
    }

    @Bean
    ServiceB serviceB() {
        return new ServiceB();
    }

    @Bean
    ServiceC serviceC() {
        return new ServiceC(serviceB());
    }
}
```

```java
@AutoConfiguration
public class MyAutoConfig {

    @Bean
    ServiceA serviceA(ServiceB serviceB) {
        return new ServiceA(serviceB);
    }

    @Bean
    ServiceB serviceB() {
        return new ServiceB();
    }

    @Bean
    ServiceC serviceC(ServiceB serviceB) {
        return new ServiceC(serviceB);
    }
}
```

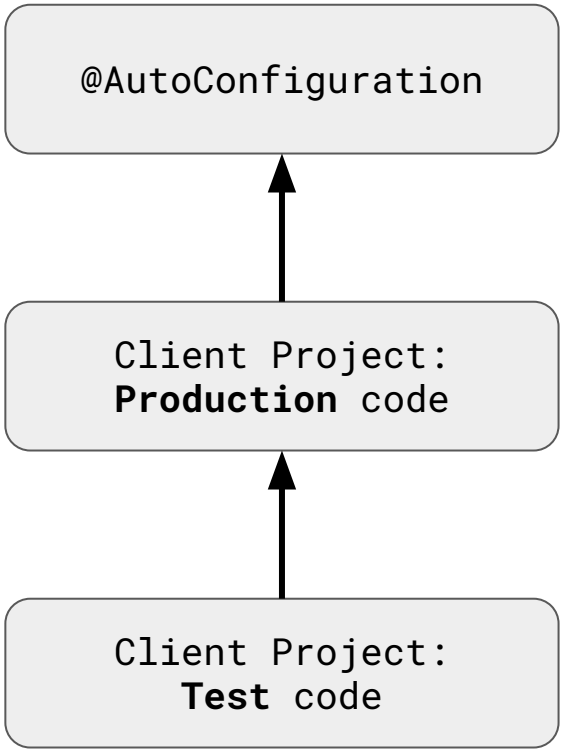
Auto Configurations

```
@AutoConfiguration
@ConditionalOnClass(Tracer.class)
@EnableConfigurationProperties(ObservabilityProperties.class)
public class MicrometerObservabilityAutoConfig {

    @Bean
    @ConditionalOnMissingBean
    ObservabilityContext micrometerObservabilityContext(Tracer tracer) {
        return new MicrometerObservabilityContext(tracer);
    }
}
```

@Conditional annotations give the possibility to easily override the beans defined in auto configurations

Overriding beans



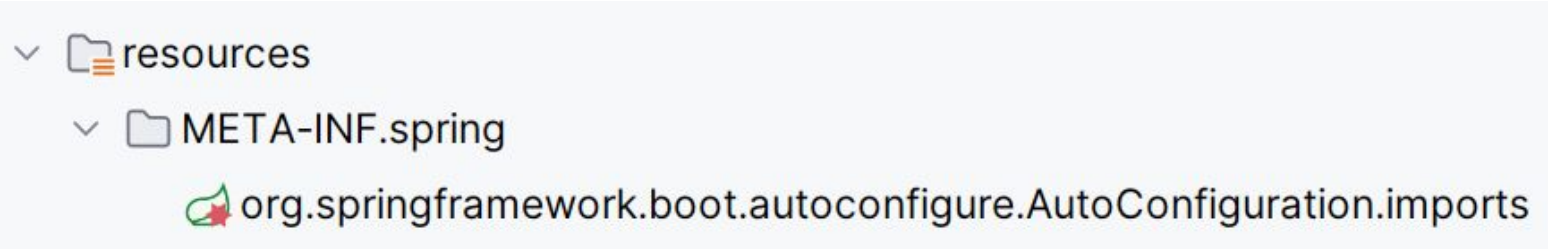
Default bean instances

Custom bean instances.
Just define them.

Testing bean instances.
Use `@Primary` annotation.

Auto Configurations

- Make sure your auto configuration runs **after** ordinary configurations!



Code quality

- To reduce maintenance costs, we need to deliver code of good quality. Ideally, automatically verified.

- In Apotek 1, we use the following tools:
  - **EditorConfig** - for code formatting
  - **Checkstyle** and **Error Prone** - to adhere to coding standards
  - **ArchUnit** - to spot barely visible issues and for knowledge sharing
  - **Snyk** - for automated vulnerability scans

trainitek

ArchUnit example: shared library

```
@ArchTest
public static final ArchRule controllersAreAnnotatedWithOpenAPIAnnotations =
        controllers().should().beAnnotatedWith(Tag.class)
        .as("Controllers should be annotated with Open API annotations");

public static ArchRule codeIsFreeOfPackageDependencyCyclesRule(String basePackage) {
    return slices().matching(basePackage + ".(**)")
            .namingSlices(basePackage + ".$1")
            .should().beFreeOfCycles()
            .as("Code is free of package dependency cycles")
            .because("package dependency cycles is an indicator of problems with
                      code design related to modularity and dependency direction.
                      Read more at https://our_wiki/…");
}
```

APOTEK 1

ArchUnit example: client project

```
class ArchitectureTest {

   @ArchTest
   public static final ArchTests standardTests = ArchTests.in(StandardArchTests.class);

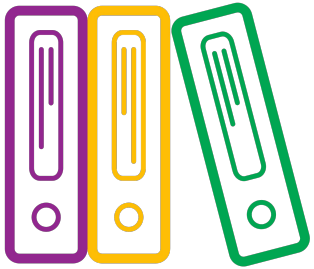
   @ArchTest
   public static final ArchTests springTests = ArchTests.in(SpringFrameworkArchTests.class);

   @ArchTest
   public static final ArchTests dddTests = ArchTests.in(DDDArchTests.class);

   @ArchTest
   public static final ArchTests cqsTests = ArchTests.in(CQSArchTests.class);

   @ArchTest
   public static final ArchTests webTests = ArchTests.in(WebArchTests.class);
}
```

Documentation



Make sure people can find it

- README files
- Confluence
- etc.

DRY?

- The DRY principle doesn’t work that well with documentation
- Sometimes, it’s worth having duplicated information, cross-links, etc. in multiple places
- This is a price to pay for your convenience

README.md - principles

- Make sure people can navigate through multiple README files by using **links**
- Imagine that your documentation is read by **junior** developers
- Supply your documentation with **examples** and **guidelines**

trainitek

README.md - example

DataHub

This module provides common APIs that can be used by

DataHub is an external highly available service that prov
organizations, prices, etc.

Usage

Maven Dependencies

In most cases, you just need to include the javalibs-data

```
<dependency>
    <groupId>no.apotek1.javalibs</groupId>
    <artifactId>javalibs-datahub-spring-boot-start
    <version>${javalibs.version}</version>
</dependency>
```

Add the test support module as well:

```
<dependency>
    <groupId>no.apotek1.javalibs</groupId>
    <artifactId>javalibs-datahub-test</artifactId>
    <version>${javalibs.version}</version>
    <scope>test</scope>
</dependency>
```

Configuration

Documentation and default values are automatically provided in the c
typing `apotek1.javalibs.datahub` in your `application.yml` or a

```
apotek1:
  javalibs:
    datahub:
      base-url: https://api-management-url
      apim-token: ${DATAHUB_APIM_TOKEN}
      timeout-sec: 30
      sync:
        token-registry: in_memory
        batch-size: 50
        iterations-warning-threshold: 1000
        max-iterations: 1500
```

Code example (sync API)

Once you've included the starter, you can inject DataHub clients into

```
public class SyncService {

    private final DataHubSyncClient syncClient;

    public void syncPharmacies() {
        syncClient.sync(Pharmacy.class, items -> {
            // Process batches of pharmacy data
            items.forEach(item -> {
                switch (item) {
                    case SyncItem.CreateOrUpdate<Pharmacy> cre
                        Pharmacy pharmacy = createOrUpdate.dat
                        // Handle create or update logic
                    }
                    case SyncItem.Delete<Pharmacy> delete -> {
                        long pharmacyId = delete.entityId();
                        // Handle delete logic
                    }
                }
            });
        });
    }
}
```

Supported Data Types

The DataHub module supports synchronization of the following data types:

- `Organization` - Organizational data
- `Pharmacy` - Pharmacy information
- `Price` - Product pricing data
- `Product` - Product information

Testing

`FakeDataHubSyncClient` is a fake implementation of `DataHubSyncClient` for use in tests. Pre-load items per type using `givenItems()` .
Types with no items configured receive an empty list.

```
def client = new FakeDataHubSyncClient()
        .givenItems(Product, [
                new SyncItem.CreateOrUpdate<>(1L, someProduct),
                new SyncItem.Delete<>(2L)
        ])
```

Pass this instance to your class under test:

```
def myComponentUnderTest = new MyComponent(client)
```

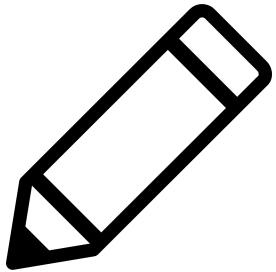
Modules

core

Core classes

APOTEK 1

Making changes



General tips

- Making changes should be straightforward for contributors
- Avoid long and complicated routines, unless you want to work alone ;)
- Fix the guidelines if people can’t follow them
- If it happens, it’s your mistake, not theirs

Versioning (3.7.10893)

5. Version 1.0.0 defines the public API. The way in which the version number is incremented after this release is dependent on this public API and how it changes.

6. Patch version Z (x.y.Z | x > 0) MUST be incremented if only backward compatible bug fixes are introduced. A bug fix is defined as an internal change that fixes incorrect behavior.

7. Minor version Y (x.Y.z | x > 0) MUST be incremented if new, backward compatible functionality is introduced to the public API. It MUST be incremented if any public API functionality is marked as deprecated. It MAY be incremented if substantial new functionality or improvements are introduced within the private code. It MAY include patch level changes. Patch version MUST be reset to 0 when minor version is incremented.

8. Major version X (X.y.z | X > 0) MUST be incremented if any backward incompatible changes are introduced to the public API. It MAY also include minor and patch level changes. Patch and minor versions MUST be reset to 0 when major version is incremented.

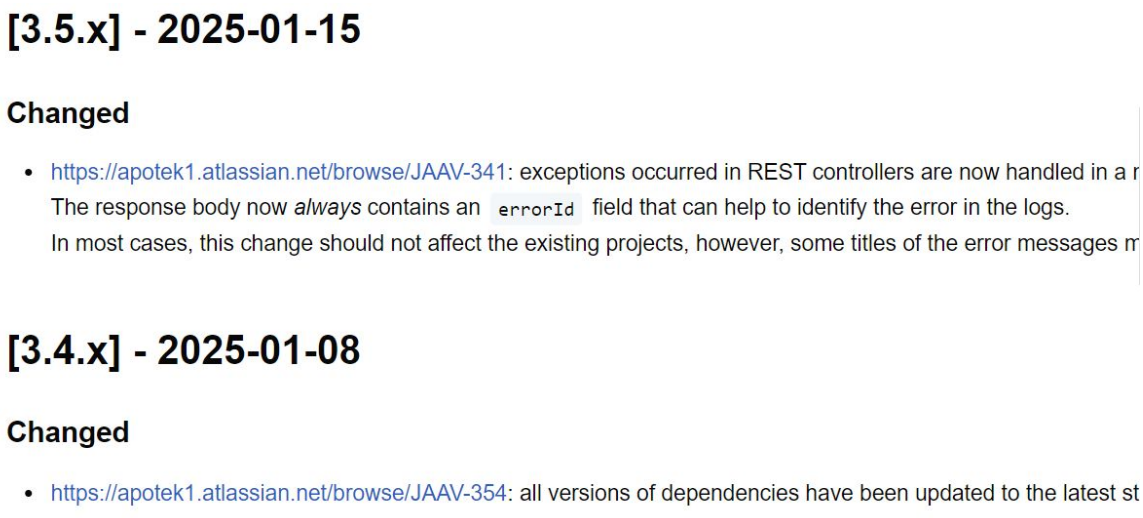
Versioning (3.7.10893)

- **Major** - for incompatible changes like:
  - JDK 21 → JDK 25
  - Upgrading major versions of Spring Framework
  - etc.
- **Minor** - for new features and ALSO for changes where some migration is required
  - We chose this because we don’t want to have too high major version (e.g. 21).
  - However, it doesn’t mean that you should do the same.
- **Patch** - for small improvements and fixes
  - This version is automatically updated by the CI/CD pipeline and is set to the build ID

trainitek

Documenting the changes

- We decided to use a **CHANGELOG** file - something simple that works for us
- https://keepachangelog.com/en/1.0.0/



[3.5.x] - 2025-01-15

Changed

- https://apotek1.atlassian.net/browse/JAAV-341: exceptions occurred in REST controllers are now handled in a r
  The response body now *always* contains an `errorId` field that can help to identify the error in the logs.
  In most cases, this change should not affect the existing projects, however, some titles of the error messages m

[3.4.x] - 2025-01-08

Changed

- https://apotek1.atlassian.net/browse/JAAV-354: all versions of dependencies have been updated to the latest st

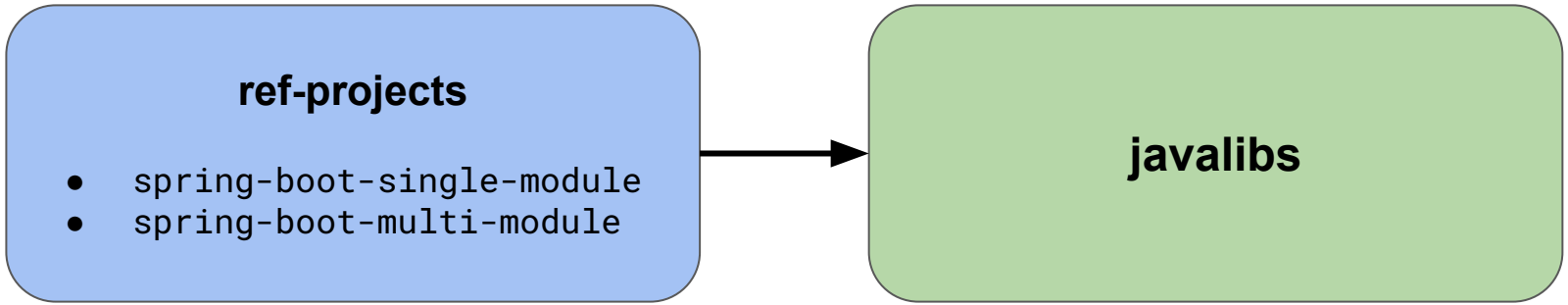
APOTEK 1

Supporting old versions

- Our shared library doesn’t support multiple major version 🤔
- Instead, we chose to constantly upgrade our systems and depend on the latest version of the library 💡
- **Result**: simplicity at the given capacity 🚀

The change is done! Just ship it?

- A successful build is not enough
- It’s smart to create an example of a client project that can help verify the changes
- In our case, we call it “reference projects”



trainitek

Reproducible builds?

```
<properties>
    <javalibs.version>[3.7,3.8)</javalibs.version>
</properties>

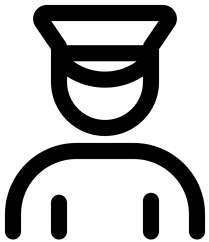
  …

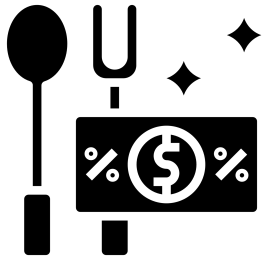
<dependency>
    <groupId
    <artifactId>javalibs-cqs-spring-boot-starter</artifactId>
    <version>${javalibs.version}</version>
</dependency>
```

Benefit: after each improvement/fix, we don't need to update every client project!

APOTEK 1

Forcing vs Advertising





Forcing vs Advertising

- Prefer to advertise common libraries rather than forcing people to use them ✅
- This is the way to verify the quality and demand for the shared code 🟡
- Otherwise, there is a chance of living inside the illusion of a good job done 🧠

Conclusions

- (1/6) Make sure you need a shared library
- (2/6) Don't include anything unnecessary
- (3/6) Create a stable and understandable structure
- (4/6) Help people navigate and contribute via well-written documentation
- (5/6) Choose the easiest versioning strategy
- (6/6) If people want to use it, you have done a good job

trainitek

Thank You!

Presentation Slides

vadim-prudnikov

APOTEK 1